

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN



Báo cáo đồ án

Đề tài: Lập trình Socket

Môn: Mạng Máy Tính

Instructor:

Chung Thụy Linh
Huỳnh Thụy Bảo Tràn

Group Members:

Dang Ba Lam - 25127403
Hua Minh Khang - 25127073
Trinh Bao Khoi - 25127256

Class: 25C10

Group ID: 07

Ho Chi Minh City, March 13, 2026

Table of Contents

1	Ma trận phân công công việc và đánh giá đồng đẳng	2
1.1	Cam kết đánh giá đồng đẳng	2
2	Bài toán, kịch bản ứng dụng và tương tác giao thức	3
2.1	Bài toán	3
2.2	Vòng đời đầy đủ của phiên	3
2.3	Liên hệ yêu cầu và mã nguồn	4
3	Kiến trúc hệ thống và cấu trúc dữ liệu	5
3.1	Phân lớp	5
3.2	Thông điệp điều khiển TCP	5
3.3	Trạng thái phiên	5
3.4	Định dạng header RDT theo byte	5
4	Thiết kế giao thức Reliable UDP	7
4.1	Bắt tay và nhận diện phiên	7
4.2	Selective Repeat	7
4.3	Cửa sổ và chống tắc nghẽn	7
4.4	Toàn vẹn và hoàn tất	7
5	Quy trình hiện thực và vận hành	8
5.1	Phân phối lệnh và concurrency	8
5.2	PORT và PASV	8
5.3	Các workflow dữ liệu	8
5.4	Điểm cần nắm khi bảo vệ	8
6	Kết quả thực nghiệm và bằng chứng demo	9
A	Nhật ký GenAI và phản biện đầu ra	12
A.1	Prompt đã sử dụng	12
A.2	Đầu ra thô	12
A.3	Audit kỹ thuật và refinement	12
A.4	Kết luận sở hữu mã	12
B	References	13

1 Ma trận phân công công việc và đánh giá đồng đẳng

Nhóm 06 – AE Siu Nhân

Các ô trong ngoặc vuông phải được thay bằng thông tin thật trước khi nộp.

STT	Họ và tên	MSSV	Mô-đun sở hữu và trách nhiệm	Tỷ lệ
1	[Thành viên 01]	[MSSV]	Kiến trúc máy chủ trong <code>server.py</code> : TCP listener, tạo một thread cho mỗi khách, <code>ClientSession</code> , xác thực, dispatcher, sandbox đường dẫn, các lệnh FTP và kiểm thử nhiều khách.	34%
2	[Thành viên 02]	[MSSV]	Lớp Reliable UDP trong <code>rdt_udp.py</code> : định dạng gói, bắt tay, Selective Repeat, ACK, timeout/truyền lại, kết thúc FIN/RESULT, CRC32 và SHA-256.	33%
3	[Thành viên 03]	[MSSV]	Khách trong <code>client.py</code> và tiện ích <code>utils.py</code> : CLI, PORT/PASV, mã phản hồi, thống kê truyền, kiểm thử khối, chụp minh chứng và tích hợp báo cáo.	33%
Tổng cộng				100%

Table 1: Phân công theo mô-đun có thể kiểm chứng khi vấn đáp

1.1 Cam kết đánh giá đồng đẳng

Ba thành viên thống nhất rằng tỷ lệ trên phản ánh trách nhiệm sở hữu, mức độ tích hợp, kiểm thử và khả năng bảo vệ trực tiếp, không chỉ số dòng mã. Mỗi người phải có thể mở đúng mô-đun của mình, giải thích luồng chạy, chỉ ra tình huống lỗi và sửa một thay đổi nhỏ khi được yêu cầu.

Thành viên	Tự đánh giá	Nhận xét đồng đẳng
hline Hứa Minh Khang	[Diễn]	Chịu trách nhiệm chứng minh cô lập phiên, luồng lệnh TCP, các mã phản hồi và an toàn đường dẫn.
Đặng Bá Lâm	[Diễn]	Chịu trách nhiệm chứng minh từng trường header, SR, <code>cwnd/ssthresh/rwnd</code> , RTO và hash đầu cuối.
Trịnh Bảo Khôi	[Diễn]	Chịu trách nhiệm chứng minh hành vi CLI, hai chế độ dữ liệu, kết quả <code>test_smoke.py</code> và bằng chứng demo.

Table 2: Đánh giá đồng đẳng cần được nhóm xác nhận trước khi nộp

2 Bài toán, kịch bản ứng dụng và tương tác giao thức

2.1 Bài toán

Hệ thống hiện thực một FTP lai: TCP duy trì mặt phẳng điều khiển, còn payload thực tế đi qua UDP với một giao thức độ tin cậy do nhóm tự xây dựng. Đây không phải FTP chuẩn RFC 959 đầy đủ; các lệnh và mã trạng thái mô phỏng quy ước FTP, trong khi kênh dữ liệu là RDT tùy biến. Cách tách này tận dụng tính có thứ tự của TCP cho xác thực/lệnh, đồng thời buộc ứng dụng tự xử lý mất gói, hỏng gói, trùng lặp và đảo thứ tự trên UDP.

Máy chủ nghe TCP mặc định tại 127.0.0.1:2121. Mỗi kết nối nhận được tạo một `ClientSession` riêng, giữ thư mục hiện thời, trạng thái đăng nhập, kiểu truyền, chế độ dữ liệu và socket UDP của chính khách đó. Các payload gồm danh sách thư mục và tệp nhị phân đều qua RDT; vì vậy LIST, NLST, RETR, STOR, STOU và APPE cùng được kiểm tra bởi cơ chế tin cậy.

2.2 Vòng đời đầy đủ của phiên

Bước	Khách		Máy chủ
1	Mở TCP; nhận banner		<code>accept()</code> , tạo thread; trả 220.
2	USER, PASS	↔	Kiểm tra; trả 331/230.
3P	PASV	→	Gắn UDP cổng tạm; trả 227 (h1,h2,h3,h4,p1,p2).
3A	Hoặc gắn UDP cục bộ, gửi PORT ...	→	Lưu địa chỉ UDP của khách; trả 200.
4	Gửi RETR, STOR hoặc LIST	→	Sinh ID 64 bit; trả 150 TRANSFER_ID=id.
5P	SYN (khách là initiator)	↔	SYN ACK (máy chủ responder).
5A	Chờ SYN (khách là responder)	↔	SYN (máy chủ initiator), nhận SYN ACK.
6	DATA/ACK, SR, <code>rwnd</code> , timeout	↔	DATA/ACK, SR, <code>rwnd</code> , timeout.
7	FIN: số byte + SHA-256	↔	FIN ACK RESULT: trạng thái + số byte + SHA-256.
8	ACK RESULT; nhận 226	↔	Đóng UDP phiên; trả 226 SHA256=... BYTES=...
9	QUIT	→	Trả 221, đóng phiên TCP.

Figure 1: Trình tự TCP + UDP; P là PASV, A là PORT

2.3 Liên hệ yêu cầu và mã nguồn

Yêu cầu mức B/A	Bằng chứng trong hiện thực
Tệp nhị phân, cây thư mục, PORT/PASV, đa khách	TYPE I; <code>safe_resolve</code> ; LIST/NLST/MKD/RMD; <code>cmd_PORT/cmd_PASV</code> ; một daemon thread cho mỗi TCP client.
RDT tự cài đặt	<code>rdt_udp.py</code> dùng duy nhất socket, <code>struct</code> , <code>zlib</code> , <code>hashlib</code> ; không dùng FTP framework hay thư viện RDT ngoài.
ACK, sequence, timeout, retransmit	<code>outstanding</code> theo sequence; ACK riêng lẻ; RTO; truyền lại chỉ các gói hết hạn.
Điều khiển luồng/tắc nghẽn	<code>peer_window</code> , <code>rwnd</code> , PROBE, <code>cwnd</code> , <code>ssthresh</code> .
Hash đầu cuối	FIN và RESULT mang byte count + SHA-256; file chỉ commit sau xác minh.

Table 3: Đối chiếu yêu cầu mức Good/Excellent với mã

3 Kiến trúc hệ thống và cấu trúc dữ liệu

3.1 Phân lớp

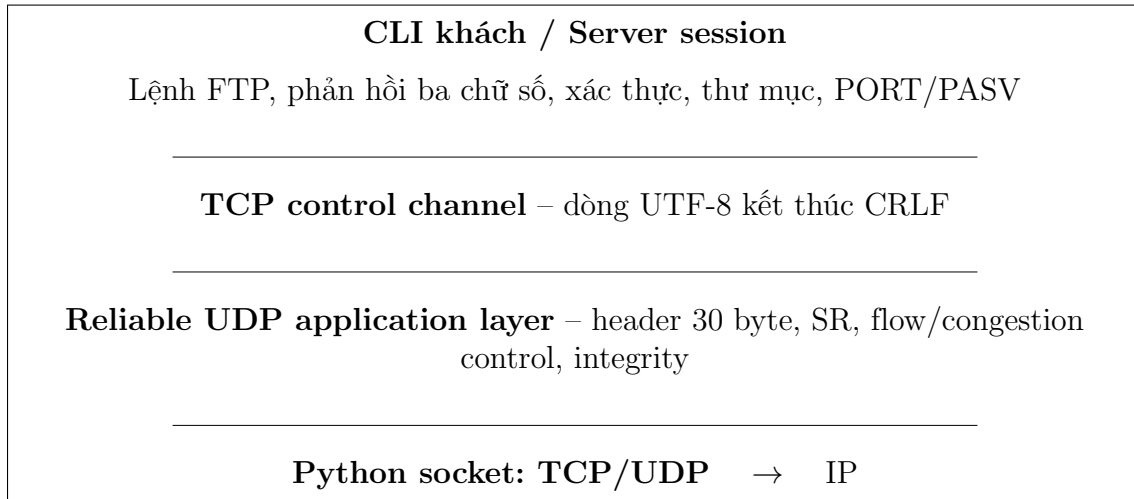


Figure 2: Tách control plane và data plane của Hybrid FTP

3.2 Thông điệp điều khiển TCP

ControlChannel không tự định nghĩa TCP header. Nó đóng khung dữ liệu ứng dụng theo dòng UTF-8: lệnh là `COMMAND [arguments]\r\n`, phản hồi là `DDD message\r\n`; mỗi dòng tối đa 8192 byte. Ví dụ, 150 Opening data channel. `TRANSFER_ID=id` liên kết lệnh TCP với phiên UDP, còn 226 Transfer complete. `SHA256=hex BYTES=n` công bố kết quả.

3.3 Trạng thái phiên

ClientSession gồm `username`, `authenticated`, `cwd`, `transfer_type`, `data_mode`, `port_addr`, `pasv_udp_sock`, `rename_from` và `abort_event`. RDTSession giữ `socket`, `peer`, `transfer_id`, trạng thái, bộ đếm gói, `cwnd`, `ssthresh`, `RTO` và `peer_window`. Các đối tượng này không được chia sẻ giữa hai phiên khách.

3.4 Định dạng header RDT theo byte

Header dùng `!4sBBQIIHHI`: big-endian, không padding, cố định 30 byte. Payload tối đa 1200 byte và toàn bộ datagram tối đa 1230 byte.

Byte	Bit	Kích thước	Trường	Diễn giải
0–3	0–31	4	MAGIC	RDT1; loại datagram không thuộc RDT.
4	32–39	1	VERSION	Giá trị 1.
5	40–47	1	flags	Bitmap loại gói.
6–13	48–111	8	transfer_id	ID ngẫu nhiên 64 bit, khác 0.
14–17	112–143	4	sequence	DATA/FIN bắt đầu từ 1; bắt tay dừng 0.
18–21	144–175	4	acknowledgment	Số DATA được ACK; 0xFFFFFFFF là không ACK.
22–23	176–191	2	rwnd	Số slot nhận còn trống, đơn vị gói.
24–25	192–207	2	payload_length	10–1200; phải khớp độ dài datagram.
26–29	208–239	4	checksum	CRC32(header với checksum bằng 0 payload).
30–	240–	0–1200	payload	Dữ liệu, metadata FIN, RESULT hoặc lý do ABORT.

Table 4: Header Reliable UDP đến mức byte/bit

Bit	Cờ	Ý nghĩa
0	0x01 SYN	Yêu cầu bắt tay: seq=0, ack=NO_ACK.
1	0x02 ACK	ACK DATA; kết hợp SYN hoặc RESULT cho điều khiển.
2	0x04 DATA	Payload dữ liệu, không được rỗng.
3	0x08 FIN	Kết thúc, payload là Q32s: số byte và SHA-256.
4	0x10 ABORT	Dừng phiên, mang lý do UTF-8.
5	0x20 RESULT	Kết hợp FIN ACK RESULT hoặc ACK RESULT.
6	0x40 PROBE	Thăm dò cửa sổ bằng không.
7	0x80	Không hợp lệ; decoder loại bỏ.

Table 5: Flags của RDT

4 Thiết kế giao thức Reliable UDP

4.1 Bắt tay và nhận diện phiên

Sau khi nhận TRANSFER_ID từ TCP, initiator gửi SYN với `sequence=0`. Responder chỉ chấp nhận SYN khi ID đúng và địa chỉ peer phù hợp, sau đó gửi SYN|ACK. Initiator retry SYN tối đa `max_retries+1` lần; RTO được nhân đôi đến tối đa 3 giây. Điều này tạo một phiên UDP logic dù UDP không có kết nối.

4.2 Selective Repeat

Máy gửi chia nguồn thành đoạn tối đa 1200 byte và gán sequence từ 1. Bảng `outstanding` giữ riêng từng gói chưa ACK. Máy nhận giữ `reorder_buffer`; gói trong cửa sổ được lưu theo sequence, còn dải liên tiếp mới được ghi xuống sink. ACK xác nhận đúng sequence đã nhận, kể cả gói đến trễ/đến lặp. Khi timeout, máy gửi retransmit *chỉ* gói đã hết hạn, không phát lại toàn bộ cửa sổ như Go-Back-N.

4.3 Cửa sổ và chống tắc nghẽn

Kích thước gửi mới bị giới hạn bởi:

$$W = \min(\max(1, \lfloor cwnd \rfloor), peer_window, max_cwnd).$$

Phía nhận quảng bá `rwnd=receive_window-reorder_buffer`. Khi `rwnd=0`, phía gửi dừng DATA mới và phát PROBE mỗi 0.50 giây. Cấu hình mặc định là `cwnd=1`, `ssthresh=16`, `max_cwnd=64`, `receive_window=64`.

ACK mới tăng `cwnd` thêm 1 dưới `ssthresh` (slow start), hoặc thêm $1/cwnd$ trên ngưỡng (congestion avoidance). Timeout đặt `ssthresh=max(2, cwnd/2)` và `cwnd=1`. RTO dùng SRTT/RTTVAR có cận 0.10–3.00 giây; mẫu RTT chỉ lấy từ gói chưa retransmit.

4.4 Toàn vẹn và hoàn tất

CRC32 phát hiện lỗi từng datagram. Hash SHA-256 giải quyết toàn vẹn end-to-end: phía gửi hash trước tệp rồi hash lại khi đọc; FIN mang số byte và digest. Phía nhận hash dữ liệu đã ghi, so sánh hai giá trị, flush sink và chỉ `os.replace()` tệp tạm khi hợp lệ. RESULT có cấu trúc BQ32s: trạng thái 1 byte, số byte 8 byte, digest 32 byte. FIN|ACK|RESULT được xác nhận bằng ACK|RESULT và phía nhận linger để xử lý FIN lặp.

Trạng thái	Chuyển tiếp	Điều kiện
hline CLOSED	SYN_SENT/SYN_RECEIVED	Không tạo hoặc nhận SYN hợp lệ.
ESTABLISHED	SENDING/RECEIVING	Gọi <code>send_*</code> hoặc <code>receive_*</code> .
SENDING	FIN_WAIT	Nguồn cạn và mọi DATA đã ACK.
RECEIVING	LINGER	Nhận FIN đúng sequence và gửi RESULT.
FIN_WAIT/LINGER	COMPLETE	Kết quả/ACK cuối hợp lệ.
Mọi trạng thái	FAILED/ABORTED	Hết retry, idle timeout, lỗi giao thức, hash sai hoặc ABORT.

Table 6: Các trạng thái RDT được dùng trong mã

5 Quy trình hiện thực và vận hành

5.1 Phân phối lệnh và concurrency

Máy chủ chạy `accept()` trong vòng lặp. [1] Mỗi TCP client được gán một thread daemon gọi `ClientSession.run()`. Vòng lặp phiên gửi 220, nhận từng dòng, kiểm tra đăng nhập, tìm handler có tên `cmd_<COMMAND>`, rồi trả 500, 501, 530, 550 hoặc mã thành công phù hợp. Các trường phiên là private cho thread đó.

`accept()` → tạo thread → `ClientSession` → gửi 220 → đọc CRLF → kiểm tra auth → dispatch handler → trả FTP code → lặp hoặc đóng

Figure 3: Luồng dispatch trên mỗi phiên TCP

5.2 PORT và PASV

Trong PASV, `cmd_PASV` tạo UDP socket của server tại cổng 0 và trả 227. Sau 150, khách gửi SYN tới socket này; server responder chỉ chấp nhận IP TCP client. Trong PORT, khách gán UDP socket, gửi sáu octet địa chỉ; server lưu `port_addr`, tạo socket UDP mới khi truyền và là initiator. Socket dữ liệu được đóng sau mỗi transfer; PASV/PORT mới cũng đóng socket PASV cũ.

5.3 Các workflow dữ liệu

1. **LIST/NLST**: server tạo bytes UTF-8 từ thư mục, mở RDT rồi gọi `send_bytes()`.
2. **RETR**: server kiểm tra tệp, gọi `send_file()`; client gọi `receive_file()` vào tệp tạm.
3. **STOR/STOU**: client gửi file bằng `send_file()`; server gọi `receive_file()`, thay tệp đích nguyên tử sau khi hash hợp lệ.
4. **APPE**: nếu tệp đích đã tồn tại, server nhận bytes vào bộ nhớ rồi mở tệp ở mode `ab`; nếu chưa có, dùng luồng nhận tệp thông thường.

5.4 Điểm cần nắm khi bảo vệ

- TYPE `A/I` chỉ lưu trạng thái và báo phản hồi; payload thực tế vẫn được xử lý ở dạng bytes, nên binary được bảo toàn.
- MODE hiện chỉ chấp nhận `S`; không tuyên bố hỗ trợ Block hoặc Compressed.
- ABOR có `abort_event`, nhưng command loop của một phiên chạy đồng bộ với data handler. Vì vậy ABOR từ chính control socket không được đọc giữa lúc transfer đang block. Đây là hạn chế phải trình bày trung thực; hướng nâng cấp là luồng đọc control riêng hoặc multiplexing.
- `test_smoke.py` chứng minh nhiều hành vi, nhưng một lần chạy PASS không thay thế việc giải thích cơ chế tại viva.

6 Kết quả thực nghiệm và bằng chứng demo

Mỗi khung dưới đây phải được thay bằng ảnh chụp/log thật của lần chạy cuối. Không dùng ảnh minh họa hoặc ảnh từ máy khác không có log của nhóm.



Figure 4: Đăng nhập và upload

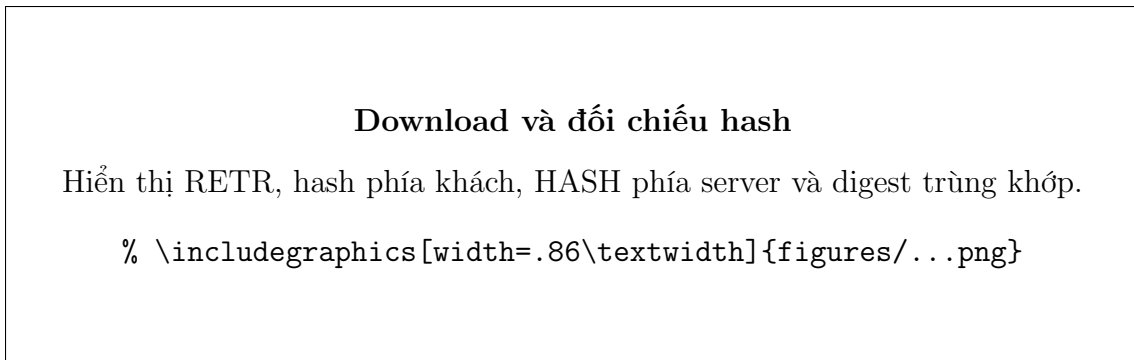


Figure 5: Download và đối chiếu hash



Figure 6: Tập nhị phân

PASV và PORT

Hiện thị 227 + transfer PASV, sau đó PORT + transfer active thành công.

```
% \includegraphics[width=.86\textwidth]{figures/...png}
```

Figure 7: PASV và PORT

Nhiều khách đồng thời

Log server có nhiều thread client-IP:port và kết quả 4 concurrent clients.

```
% \includegraphics[width=.86\textwidth]{figures/...png}
```

Figure 8: Nhiều khách đồng thời

Kết quả test khói

Ảnh terminal cuối test_smoke.py, gồm tổng PASS/FAIL.

```
% \includegraphics[width=.86\textwidth]{figures/...png}
```

Figure 9: Kết quả test khói

Ca	Tiêu chí quan sát	Kết quả cuối
hline Authentication	Admin/anonymous hợp lệ; sai user/password và lệnh trước login bị từ chối.	<i>[PASS/FAIL]</i>
Directory	Navigation, nested directory, traversal confinement.	<i>[PASS/FAIL]</i>
PASV/PORT	LIST, STOR, RETR thành công ở cả hai mode.	<i>[PASS/FAIL]</i>
Integrity	Hash SHA-256 và bytes khớp cho tệp binary.	<i>[PASS/FAIL]</i>
Concurrency	Bốn khách PWD/LIST độc lập.	<i>[PASS/FAIL]</i>

Table 7: Bảng kết quả cần cập nhật từ lần chạy thực

A Nhật ký GenAI và phản biện đầu ra

Yêu cầu trung thực: phần này là khung mẫu. Nhóm phải thay prompt và raw output bằng bản ghi chính xác của chính nhóm; không được gắn nhãn một nội dung tái tạo là “đầu ra thô”.

A.1 Prompt đã sử dụng

[Dán nguyên văn prompt thực tế, gồm bối cảnh, ràng buộc và câu hỏi.]

A.2 Đầu ra thô

[Dán nguyên văn output chưa chỉnh sửa; có thể rút gọn phần không liên quan, nhưng phải ghi rõ phần đã lược.]

A.3 Audit kỹ thuật và refinement

Gợi ý AI	Vấn đề/rủi ro	Quyết định của nhóm và bằng chứng mã
hline ACK cộng dồn hoặc retransmit cả cửa sổ	Không mô tả đúng SR; lãng phí băng thông.	Nhóm dùng ACK theo sequence và outstanding ; timeout chỉ retransmit gói hết hạn.
FIN đơn giản	Không đủ xác nhận digest, không xử lý mất kết quả.	FIN chứa Q32s ; RESULT là BQ32s ; có ACK RESULT và linger .
Chỉ nói “dùng checksum”	Không phân biệt hồng datagram và toàn vẹn tệp.	CRC32 cho packet; SHA-256 + số byte end-to-end; commit tệp sau verify.
Không nói zero window	Có thể deadlock khi receiver quảng bá 0.	PROBE định kỳ và ACK mang rwnd .

Table 8: Ví dụ về phản biện phải được đối chiếu với prompt/output thật

A.4 Kết luận sở hữu mã

Nhóm đã rà lại từng đề xuất trước khi tích hợp: định dạng **HEADER**, xử lý **send_stream/receive_stream**, hàm tăng/giảm cửa sổ và **FIN/RESULT**. Nhóm cũng ghi nhận giới hạn **ABOR** đồng thời thay vì tuyên bố một tính năng không được chứng minh bởi luồng thực thi hiện tại. Mỗi thành viên cam kết có thể giải thích phần mã mình sở hữu trong vấn đáp.

B References

References

- [1] Tarundeep Singh Sodhi, Surmeet Kaur, and Snehdeep Kaur. Enhanced insertion sort algorithm. *International journal of Computer applications*, 64(21):35–39, 2013.